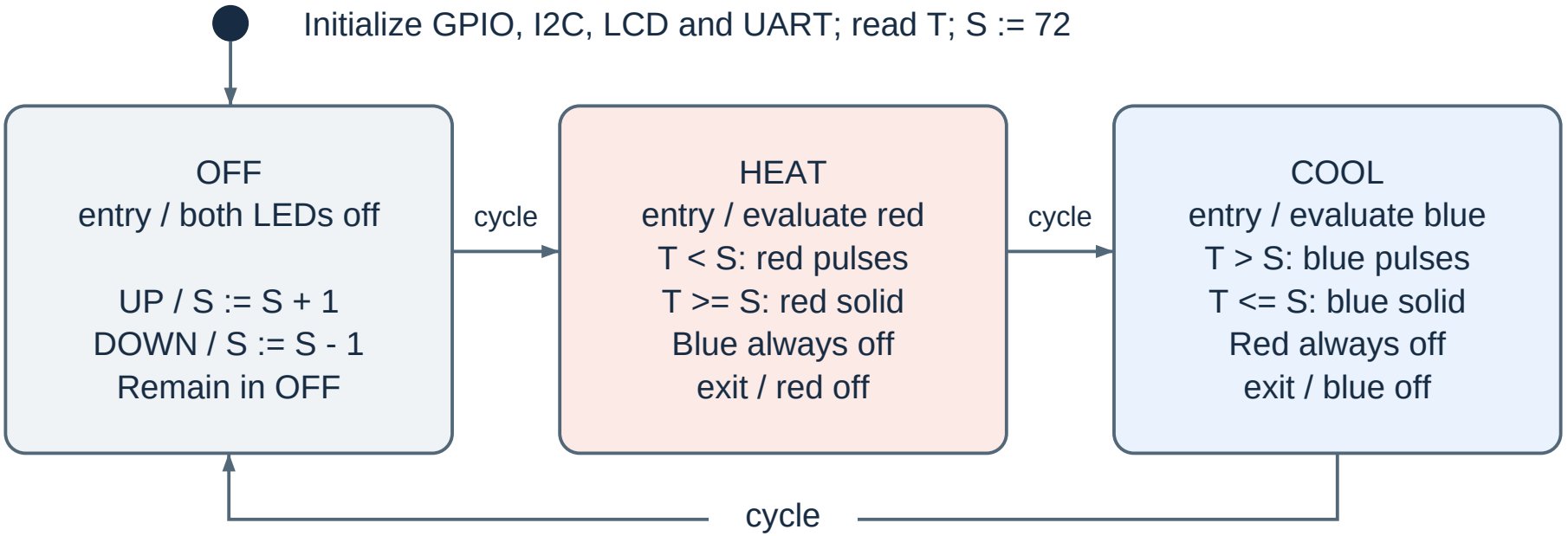
CS 350 Thermostat State Machine

Thermostat.py | UML state notation | T = current whole-degree F; S = set point F

cycle = GPIO 24 press UP = GPIO 25 press DOWN = GPIO 12 press



Internal events in HEAT and COOL (mode stays unchanged):
UP / S := S + 1; evaluate indicators
DOWN / S := S - 1; evaluate indicators
sample / refresh T; evaluate indicators

Solid = selected mode satisfied.
Pulse = active heating/cooling.
At T = S, the selected LED is solid.
No automatic HEAT/COOL change.

Any mode: Ctrl+C or fatal I/O error / both LEDs off; close UART; clear LCD; release pins

Notation: event [guard] / action. Conditions in state boxes select the indicator action.

Thermostat Interfaces and Timing

Technical specification | All GPIO numbers use BCM numbering

I2C temperature input
AHT20 at 0x38; SDA GPIO 2; SCL GPIO 3.
board.I2C() then adafruit_ahtx0.AHTx0(i2c).
Read Celsius at startup and once per second.
T = floor(C x 9/5 + 32); LCD, control and UART
use this same whole-degree Fahrenheit value.

GPIO inputs and indicators
Buttons: mode 24; increase 25; decrease 12.
Active-low; pull-ups; 50 ms debounce.
when_pressed queues one event per press.
LEDs: red 18; blue 23; PWMLED control.
Fade: 1 s in + 1 s out; inactive color is off.

LCD output
16 x 2; RS/EN/D4/D5/D6/D7 use GPIO
17 / 27 / 5 / 6 / 13 / 26, respectively.
Row 1 always: MM/DD HH:MM:SS (24-hour).
Row 2: Temp: 70F for 5 s, then
HEAT Set:72F for 5 s; repeats in every mode.
Rows are padded; update each second or press.

UART output
/dev/ttyS0 (configurable with --port).
115200 baud; 8 data bits; no parity; 1 stop bit.
No flow control; 1 s read/write timeouts.
TX GPIO 14; RX GPIO 15; common ground.
Every 30 s: state,T,S plus newline (UTF-8).
Example: heat,70,72\n; first send at t = 30 s.

Execution and timing

GPIO Zero event callbacks enqueue presses; the main loop handles each event in order.
The main loop also owns the sensor, LCD and UART. PWM fades run in the background.
Monotonic deadlines preserve the 1 s and 30 s schedules; LCD page changes use elapsed time.
Late service skips missed timer slots without sending a burst of old records.
Button presses never reset deadlines. Fatal I/O errors stop execution and release resources.